04-02. Shred Fetch Stage 구현 분석

개요

Shred Fetch Stage는 TVU 파이프라인의 첫 번째 단계로, 네트워크에서 shred(블록 데이터 조각)를 수신하고 기본적인 필터링을 수행하는 역할을 담당합니다. 이 단계는 UDP 소켓과 QUIC 프로토콜을 통해 다양한 소스로부터 shred를 수집하고, 유효하지 않은 shred를 조기에 필터링하여 후속 단계의 부하를 줄입니다.

ShredFetchStage 구조체

```
pub(crate) struct ShredFetchStage {

thread_hdls: Vec<JoinHandle<()>>,

}
```

ShredFetchStage는 여러 개의 스레드 핸들을 관리하는 단순한 구조체입니다. 각 스레드는 서로 다른 네트워크 소스나 처리 단계를 담당합니다.

주요 상수 및 설정

```
const PACKET_COALESCE_DURATION: Duration = Duration::from_millis(1);

const RECV_TIMEOUT: Duration = Duration::from_secs(1);
```

- `PACKET_COALESCE_DURATION` : 패킷 병합 시간 (1ms)
- `RECV_TIMEOUT` : 수신 타임아웃 (1초)

네트워크 수신 메커니즘

1. UDP 소켓을 통한 Shred 수신

ShredFetchStage는 여러 개의 UDP 소켓을 통해 shred를 수신합니다:

```
pub(crate) fn  new(
sockets: Vec<Arc<UdpSocket>>, // 일반 shred 수신용 소켓들
turbine_quic_endpoint_receiver: Receiver<(Pubkey, SocketAddr, Bytes)>,
repair_socket: Arc<UdpSocket>, // Repair 전용 소켓
repair_quic_endpoint_receiver: Receiver<(SocketAddr, Vec<u8>)>,
sender: Sender<PacketBatch>,
shred_version: u16,
bank_forks: Arc<RwLock<BankForks>>,
cluster_info: Arc<ClusterInfo>,
turbine_disabled: Arc<AtomicBool>,
exit: Arc<AtomicBool>,
) -> Self
```

2. 다중 수신 채널 구성

ShredFetchStage는 다음과 같은 여러 채널을 통해 shred를 수신합니다:

1. **일반 TVU 소켓들**: 정상적인 블록 전파를 위한 shred 수신
2. **Repair 소켓**: 누락된 shred 복구를 위한 전용 채널
3. **QUIC Turbine**: QUIC 프로토콜을 통한 고성능 shred 전송

4. **QUIC Repair**: QUIC 프로토콜을 통한 repair shred 수신

패킷 처리 파이프라인

packet_modifier 함수

각 수신 채널에 대해 `packet_modifier` 함수가 다음과 같은 구조로 패킷 처리 파이프라인을 생성합니다:

```
fn packet_modifier(
receiver_thread_name: &'static str,
modifier_thread_name: &'static str,
sockets: Vec<Arc<UdpSocket>>,
exit: Arc<AtomicBool>,
sender: Sender<PacketBatch>,
recycler: PacketBatchRecycler,
bank_forks: Arc<RwLock<BankForks>>,
shred_version: u16,
name: &'static str,
flags: PacketFlags,
repair_context: Option<(Arc<UdpSocket>, Arc<ClusterInfo>)>,
turbine_disabled: Arc<AtomicBool>,
) -> (Vec<JoinHandle<()>>, JoinHandle<()>)
```

이 함수는 다음과 같은 구조를 생성합니다:

1. **Receiver 스레드들**: 각 소켓에서 원시 패킷을 수신

2. **Modifier 스레드**: 수신된 패킷을 필터링하고 처리

modify_packets 함수 - 핵심 필터링 로직

```
fn modify_packets(
recvr: PacketBatchReceiver,
sendr: Sender<PacketBatch>,
bank_forks: &RwLock<BankForks>,
shred_version: u16,
name: &'static str,
flags: PacketFlags,
repair_context: Option<(&UdpSocket, &ClusterInfo)>,
turbine_disabled: Arc<AtomicBool>,
)
```

이 함수는 다음과 같은 필터링 과정을 수행합니다:

1. 동적 상태 업데이트

```
if last_updated.elapsed().as_millis() as u64 > DEFAULT_MS_PER_SLOT {
last_updated = Instant::now();
let root_bank = {
let bank_forks_r = bank_forks.read().unwrap();
last_slot = bank_forks_r.highest_slot();
bank_forks_r.root_bank()
```

```
};

feature_set = root_bank.feature_set.clone();

epoch_schedule = root_bank.epoch_schedule().clone();

last_root = root_bank.slot();

slots_per_epoch = root_bank.get_slots_in_epoch(root_bank.epoch());

}
```

슬롯당 기본 시간(DEFAULT_MS_PER_SLOT)마다 다음 정보를 업데이트합니다:

- 현재 루트 슬롯
- 에포크당 슬롯 수
- 기능 세트 (feature set)
- 에포크 스케줄

2. Repair 응답 핑 처리

```
if let Some((udp_socket, _)) = repair_context {

debug_assert_eq!(flags, PacketFlags::REPAIR);

debug_assert!(keypair.is_some());

if let Some(ref keypair) = keypair {

ServeRepair::handle_repair_response_pings(

udp_socket,

keypair,

&mut packet_batch,

&mut stats,

);
```

```
}
}
```

Repair 컨텍스트가 있는 경우, repair 응답 핑을 처리합니다.

3. Shred 유효성 검사 및 필터링

```
let max_slot = last_slot + 2 * slots_per_epoch;
let should_drop_legacy_shreds = |shred_slot| {
check_feature_activation(
&feature_set::drop_legacy_shreds::id(),
shred_slot,
&feature_set,
&epoch_schedule,
)
};
let enable_chained_merkle_shreds = |shred_slot| {
check_feature_activation(
&feature_set::enable_chained_merkle_shreds::id(),
shred_slot,
&feature_set,
&epoch_schedule,
)
};
```

```
for packet in packet_batch.iter_mut().filter(|p| !p.meta().discard()) {
if turbine_disabled
|| should_discard_shred(
packet,
last_root,
max_slot,
shred_version,
should_drop_legacy_shreds,
enable_chained_merkle_shreds,
&mut stats,
)
{
packet.meta_mut().set_discard(true);
} else {
packet.meta_mut().flags.insert(flags);
}
}
```

각 패킷에 대해 다음 검사를 수행합니다:

- Turbine 비활성화 여부 확인
- `should_discard_shred` 함수를 통한 상세 검증
- 유효한 shred에 적절한 플래그 설정

Shred 폐기 조건 (should_discard_shred)

`should_discard_shred` 함수는 다음과 같은 조건에서 shred를 폐기합니다:

1. **인덱스 오버런**: 패킷 크기가 너무 작아 shred 인덱스를 읽을 수 없는 경우
2. **슬롯 범위 초과**:
   - 슬롯이 루트 슬롯보다 작은 경우
   - 슬롯이 최대 허용 슬롯(현재 슬롯 + 2 에포크)보다 큰 경우
3. **Shred 버전 불일치**: 네트워크 shred 버전과 일치하지 않는 경우
4. **인덱스 범위 초과**: Shred 인덱스가 최대 허용값을 초과하는 경우
5. **레거시 Shred**: 기능 활성화에 따른 레거시 shred 폐기

QUIC 프로토콜 지원

Turbine QUIC 수신

```
fn receive_quic_datagrams(
turbine_quic_endpoint_receiver: Receiver<(Pubkey, SocketAddr, Bytes)>,
sender: Sender<PacketBatch>,
recycler: PacketBatchRecycler,
exit: Arc<AtomicBool>,
)
```

QUIC을 통한 turbine shred 수신:

- 높은 처리량과 낮은 지연시간 제공

- 패킷 병합을 통한 효율성 향상
- 송신자 공개키 정보 포함

Repair QUIC 수신

```
pub(crate) fn receive_repair_quic_packets(
repair_quic_endpoint_receiver: Receiver<(SocketAddr, Vec<u8>)>,
sender: Sender<PacketBatch>,
recycler: PacketBatchRecycler,
exit: Arc<AtomicBool>,
)
```

QUIC을 통한 repair shred 수신:

- 신뢰성 있는 repair 데이터 전송
- 자동으로 REPAIR 플래그 설정

메모리 관리 최적화

PacketBatch 재활용

```
let recycler = PacketBatchRecycler::warmed(100, 1024);
```

ShredFetchStage는 PacketBatch 재활용을 통해 메모리 할당 오버헤드를 최소화합니다:

- 100개의 사전 할당된 PacketBatch
- 각 배치당 1024개의 패킷 용량

패킷 병합 (Packet Coalescing)

```
let deadline = Instant::now() + PACKET_COALESCE_DURATION;
let entries = std::iter::once(entry).chain(
std::iter::repeat_with(|| turbine_quic_endpoint_receiver.recv_deadline(deadline)
.while_some(),
);
```

1ms 동안 여러 패킷을 병합하여 처리 효율성을 향상시킵니다.

성능 모니터링

ShredFetchStats

ShredFetchStage는 다음과 같은 통계를 수집합니다:

- `shred_count` : 수신된 총 shred 수
- `index_overrun` : 인덱스 오버런 발생 횟수
- `slot_out_of_range` : 슬롯 범위 초과 횟수
- `shred_version_mismatch` : 버전 불일치 횟수

통계 제출

```
stats.maybe_submit(name, STATS_SUBMIT_CADENCE);
```

1초마다 통계를 제출하여 시스템 모니터링을 지원합니다.

스레드 구성

ShredFetchStage는 다음과 같은 스레드들을 생성합니다:

1. **solRcvrShred##**: 일반 shred 수신 스레드들 (소켓 수만큼)
2. **solTvuPktMod**: 일반 shred 패킷 수정 스레드
3. **solRcvrShredRep**: Repair shred 수신 스레드
4. **solTvuRepPktMod**: Repair shred 패킷 수정 스레드
5. **solTvuRecvRpr**: QUIC repair 패킷 수신 스레드
6. **solTvuFetchRpr**: QUIC repair 패킷 처리 스레드
7. **solTvuRecvQuic**: QUIC turbine 패킷 수신 스레드
8. **solTvuFetchQuic**: QUIC turbine 패킷 처리 스레드

기능 활성화 검사

```
fn  check_feature_activation(
feature: &Pubkey,
shred_slot: Slot,
feature_set: &FeatureSet,
epoch_schedule: &EpochSchedule,
) -> bool
```

이 함수는 특정 기능이 주어진 shred 슬롯에 대해 활성화되어 있는지 확인합니다:

- 기능 활성화 슬롯을 확인
- 에포크 기반으로 기능 적용 여부 결정

- 이전 에포크에서 활성화된 기능만 현재 에포크에 적용

ShredFetchStage는 Solana의 고성능 블록체인 처리를 위한 첫 번째 관문 역할을 하며, 효율적인 네트워크 수신과 초기 필터링을 통해 전체 시스템의 성능을 최적화합니다.